

IN5020 Assignment 1 Report

Group 8 IN5020 - Distributed Systems

Members:

borgebj | Børge

vegarhje | Vegard

sellebaf | Selleban

Design and interface

When designing our solution we made it very clear that we wanted to stick to the assignment description as closely as possible. That being said some parts of the Client-Server solution descriptions were vague leading to us making decision based on our interpretation.

In our server-client solution, we closely followed the task description while adding several supplementary classes, primarily located in the "data" folder, to better modularize and separate the code.

We created two interfaces: one for client-proxy communication and another for server-proxy communication.

Request handling on the server operates by inserting the requests into a queue upon receipt, thereby blocking further execution through the use of a lock. This lock ensures that the client receives the requested information as soon as it becomes available.

The execution thread within the servers initiates concurrently with the server startup. It continuously attempts to extract "Request" objects from the server's internal FIFO queue. If polling from the queue is successful, the server proceeds to compute the request and signals the lock to release.

Within the "Request" object, local information such as a key identifier, the computation task, and the associated lock is contained. Requests are created upon an RMI call and are subsequently placed in the queue.

The proxy is responsible for load balancing by verifying, with each call, the availability of the requested server relative to its workload. The proxy maintains an internal map indicating the number of requests made to each server. This information is occasionally used to fetch and balance the workload from busy servers.

The main classes of our solution are "ClientSimulator" and "ServerSimulator." The "ClientSimulator" was initially designed to run multiple clients but has now been reduced to handle a single client. The "ServerSimulator" is responsible for starting and creating both the proxy and the servers.

Optionally, one can use the "Starter" class for an easy one-instance run. This class calls both simulators with the appropriate parameters, provided through arguments when executed.

Screenshots

```
IN5020-Distribuerte-Systemer\ass1 on ♦ main [!$] via 🌐 v17.0.10
> java -cp ./target/solution.jar ass1.Starter client 20

[ Client cache used, line delay 20 ms]
proxy:1098 has started
server1:1100 has started
server2:1101 has started
server3:1102 has started
server4:1103 has started
server5:1104 has started
Client:1104 has started

Requesting zone 4 (Workload: 0)
Using requested zone: 4 (Workload: 0)
Executor [server 4] processing : request={getPopulationofCountry[French Guiana] ; 0}
Finished getPopulationofCountry[French Guiana] = 189578
189578 getPopulationofCountry French Guiana Zone:4 (turnaround time: 197 ms, execution time: 89 ms, waiting time: 108, processed by server 4)

Requesting zone 1 (Workload: 0)
Using requested zone: 1 (Workload: 0)
Executor [server 1] processing : request={getPopulationofCountry[Seychelles] ; 0}
Finished getPopulationofCountry[Seychelles] = 48029
48029 getPopulationofCountry Seychelles Zone:1 (turnaround time: 177 ms, execution time: 83 ms, waiting time: 94, processed by server 1)

Requesting zone 1 (Workload: 1)
Using requested zone: 1 (Workload: 1)
Executor [server 1] processing : request={getPopulationofCountry[Canada] ; 0}
Finished getPopulationofCountry[Canada] = 31048782
31048782 getPopulationofCountry Canada Zone:1 (turnaround time: 172 ms, execution time: 83 ms, waiting time: 89, processed by server 1)

Requesting zone 2 (Workload: 0)
Using requested zone: 2 (Workload: 0)
Executor [server 2] processing : request={getPopulationofCountry[Equatorial Guinea] ; 0}
Finished getPopulationofCountry[Equatorial Guinea] = 447039
447039 getPopulationofCountry Equatorial Guinea Zone:2 (turnaround time: 180 ms, execution time: 83 ms, waiting time: 97, processed by server 2)
447039 getPopulationofCountry Equatorial Guinea Zone:2 (turnaround time: 0 ms, execution time: 0 ms, waiting time: 0, processed by server 2)
```

Screenshot of program ran with client cache, starter program runs two threads, the running of servers and the client

```
Requesting zone 4 (Workload: 23)
Redirecting to zone:5 (Workload: 13)
Executor [server 5] processing : request={getPopulationofCountry[Tuvalu] ; 0}
Finished getPopulationofCountry[Tuvalu] = 8249
8249 getPopulationofCountry Tuvalu Zone:4 (turnaround time: 258 ms, execution time: 83 ms, waiting time: 175, processed by server 5)

Requesting zone 5 (Workload: 13)
Using requested zone: 5 (Workload: 13)
Executor [server 5] processing : request={getPopulationofCountry[Japan] ; 0}
Finished getPopulationofCountry[Japan] = 95951597
95951597 getPopulationofCountry Japan Zone:5 (turnaround time: 167 ms, execution time: 82 ms, waiting time: 85, processed by server 5)
95951597 getPopulationofCountry Japan Zone:5 (turnaround time: 0 ms, execution time: 0 ms, waiting time: 0, processed by server 5)

Requesting zone 1 (Workload: 16)
Using requested zone: 1 (Workload: 16)
Executor [server 1] processing : request={getPopulationofCountry[Guam] ; 0}
Finished getPopulationofCountry[Guam] = 184225
184225 getPopulationofCountry Guam Zone:1 (turnaround time: 167 ms, execution time: 82 ms, waiting time: 85, processed by server 1)

Requesting zone 2 (Workload: 13)
Using requested zone: 2 (Workload: 13)
Executor [server 2] processing : request={getPopulationofCountry[Botswana] ; 0}
Finished getPopulationofCountry[Botswana] = 1073335
1073335 getPopulationofCountry Botswana Zone:2 (turnaround time: 169 ms, execution time: 83 ms, waiting time: 86, processed by server 2)

Requesting zone 2 (Workload: 14)
Using requested zone: 2 (Workload: 14)
Executor [server 2] processing : request={getPopulationofCountry[Guadeloupe] ; 0}
Finished getPopulationofCountry[Guadeloupe] = 374575
374575 getPopulationofCountry Guadeloupe Zone:2 (turnaround time: 172 ms, execution time: 84 ms, waiting time: 88, processed by server 2)

Requesting zone 4 (Workload: 24)
Redirecting to zone:5 (Workload: 14)
Executor [server 5] processing : request={getPopulationofCountry[Oman] ; 0}
Finished getPopulationofCountry[Oman] = 2579609
2579609 getPopulationofCountry Oman Zone:4 (turnaround time: 274 ms, execution time: 82 ms, waiting time: 192, processed by server 5)
2579609 getPopulationofCountry Oman Zone:4 (turnaround time: 0 ms, execution time: 0 ms, waiting time: 0, processed by server 4)
2579609 getPopulationofCountry Oman Zone:4 (turnaround time: 0 ms, execution time: 0 ms, waiting time: 0, processed by server 4)
```

Screenshot showing Instances in which workload is offloaded to another server than one requested

Workload and members contribution

The dividing of work initially starting with each member being assigned the task of building the start of a larger components in our solution. These parts being: Interface, Server/Client set up, Connection, Java RMI invoking and Caching. Once these major components were implemented, the solution was iteratively refined. Any available group members would take on the task of enhancing and improving the solution by adding features and requirements such as threading, testing, server workload balancing, and writing data to files. These tasks were sent of as issues on our shared github repistory to then be taken on be whomever was willing.

User Guide

A detailed ReadMe is contained within the zipped folder that was delivered. Within it, all the necessary information needed to properly compile an run the Server-Client solution is provided.